SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. INTRODUCTION

1.1 Purpose

This document describes the functional and non-functional requirements of the **Web-Based Subscription Delivery System with Route Optimisation**. It serves as a comprehensive reference for developers, testers, and stakeholders involved in the project.

1.2 Scope

The system provides an online platform where:

- **Customers** can subscribe to products, manage subscriptions, make payments, and track deliveries.
- **Delivery Partners** receive optimised routes for efficient delivery.
- **Administrators** manage users, schedules, and generate reports.

The system aims to improve delivery efficiency through automated subscription handling and route optimisation.

1.3 Definitions, Acronyms, Abbreviations

(Reserved for future use)

1.4 References

(Reserved for future use)

1.5 Overall Description

The system is designed as a modular web application to facilitate the end-to-end lifecycle of subscription-based services, ensuring seamless interaction between users and logistics.

2. OVERALL DESCRIPTION

2.1 Product perspective

The system is a web-based application that interacts with:

- Customers
- Delivery Partners
- Admin
- Payment Gateway

It maintains centralised databases for users, subscriptions, orders, payments, and reports.

2.2 Product functions

- User registration and authentication
- Subscription management
- Order processing
- Route optimisation
- Payment processing
- Report generation
- Salary calculation

2.3 User characteristics

Customer

- **Abilities:** Able to use web browser and mobile; able to make online payments.
- **What he does:** Register and login; subscribe to a plan; place order and payment; track delivery.
- **Access:** Operations on his own account and personal data.

Delivery Partner

- **Abilities:** Basic knowledge about web application and mobile usage; able to use GPS navigation; able to update delivery status.
- **What he does:** Login; view assigned route; update delivery status; delivery confirmation.
- **Access:** View assigned delivery and his earnings.

Admin

- **Abilities:** Knowledge about system usage; understanding of database and reporting; knowledge of business operations.
- **What he does:** Manage users, delivery, schedule, and slots; generate salaries and reports.
- **Access:** Full system access.

2.4 User constraints

- **Customer:** Must register before accessing services; must provide valid information; must complete payment before activation; can only access own data; cannot modify system settings; must have internet connectivity.
- **Delivery Partner:** Must login before accessing routes; can view only assigned deliveries; cannot modify order details; must update status accurately; must use GPS-enabled device.
- **Admin:** Must authenticate before access; actions must follow role-based access control; cannot access raw payment gateway credentials; must follow data privacy policies and ensure data integrity.

2.5 Assumptions & dependencies

- **Assumptions:** Internet connection is stable; user information is valid; delivery partner has GPS enabled; payment gateway is functional.
- **Dependencies:** Depends on third-party payment gateway for payments; depends on GPS devices; depends on web server, browser, and database (MongoDB).

2.6 Apportioning requirements

(Reserved for future use: requirements that may be delayed until future versions)

3. SPECIFIC REQUIREMENTS

3.1 Interface requirements

3.1.1 External Interface

For user, delivery partner, and admin, the system requires the following external interfaces:

- Dashboards for user and delivery partner.
- Login and Register page.
- Admin panel.
- Delivery tracking page.

3.1.2 Hardware Interface

- GPS location.
- Server machine.
- Storage.

3.1.3 Software Interface

- Payment gateway APIs.
- Map APIs.
- Database (MongoDB).
- Web server.
- Browser.

3.1.4 Communication Interface

- HTTP and HTTPS protocols.
- Web Sockets.
- Restful APIs (client and server communication).
- CRUD (Database communication).
- Internet communication.

3.2 Functional requirements

3.2.1 Use case model / Information Flows

Actors: The system has three main actors: Customer, Delivery Partner, and Admin.

Customer – Use Cases:

- **Use Cases:** Register, Login, Select Subscription Plan, Manage Subscription (Modify/Pause/Cancel), Make Payment, Track Delivery, View Order Status.
- **Customer → System:** Registration data, Login credentials, Subscription details, Payment information, Delivery tracking request.
- **System → Customer:** Authentication confirmation, Subscription confirmation, Payment confirmation, Delivery status, Notifications.

Delivery Partner – Use Cases:

- **Use Cases:** Login, View Assigned Route, Update Delivery Status, Confirm Delivery, View Earnings.
- **Delivery Partner → System:** Login credentials, Delivery updates, Delivery confirmation.
- **System → Delivery Partner:** Assigned route, Delivery list, Earnings details.

Admin – Use Cases:

- **Use Cases:** Login, Manage Users, Manage Delivery Partners, Manage Schedule and Time Slots, Generate Reports, Calculate Salaries.
- **Admin → System:** User management data, Delivery partner management data, Schedule updates, Report requests, Salary requests.
- **System → Admin:** User records, Delivery partner records, Reports, Salary details.

3.2.2 Use Case Specifications/ Process description

Use Case 1: Register User

- **Preconditions:** User is not already registered.
- **Main Flow:** Customer enters details → System validates → System stores user data → System confirms registration.
- **Postconditions:** User account is created; Customer can log in.

Use Case 2: User Login

- **Preconditions:** User must be registered.
- **Main Flow:** User enters credentials → System verifies → System grants access based on role.
- **Postconditions:** User is authenticated; Dashboard is displayed.

Use Case 3: Create Subscription

- **Preconditions:** Customer must be logged in.
- **Main Flow:** Customer selects plan → Customer selects delivery slot → System processes payment → System activates subscription.
- **Postconditions:** Subscription is stored; Order generation begins.

Use Case 4: Manage Subscription

- **Preconditions:** Active subscription exists.
- **Main Flow:** Customer selects modify/pause/cancel option → System updates details → System confirms update.
- **Postconditions:** Subscription details updated in database.

Use Case 5: Process Order

- **Preconditions:** Subscription must be active.
- **Main Flow:** System creates order → Order details are stored → Delivery information forwarded.
- **Postconditions:** Order is ready for delivery.

Use Case 6: Route Optimization

- **Preconditions:** Orders available for delivery.
- **Main Flow:** System collects addresses → Applies optimization algorithm → Generates route → Assigned to partner.
- **Postconditions:** Delivery partner receives route details.

Use Case 7: Update Delivery Status

- **Preconditions:** Route must be assigned.
- **Main Flow:** Partner marks order as delivered → System updates record → Customer notified.
- **Postconditions:** Status updated in database.

Use Case 8: Payment Processing

- **Preconditions:** Valid subscription request exists.
- **Main Flow:** Customer enters details → Request sent to gateway → Payment verified → Confirmation generated.
- **Postconditions:** Payment record stored; Subscription activated.

Use Case 9: Generate Reports

- **Preconditions:** Admin must be logged in.
- **Main Flow:** Admin selects report type → System retrieves data → Report generated and displayed.
- **Postconditions:** Report stored in database.

Use Case 10: Calculate Salary

- **Preconditions:** Delivery records must exist.
- **Main Flow:** System collects delivery data → System calculates salary → Salary details displayed.
- **Postconditions:** Salary record is stored.

3.2.3 Analysis Classes/Data Dictionary

(Reserved for future use)

3.3 Performance requirements

- Support at least **1000 concurrent users**.
- Average response time shall be **less than 3 seconds** for user actions.
- Payment processing shall be completed within **5 seconds**.
- Route optimization shall generate routes within **10 seconds**.
- Handle simultaneous subscription and order requests without failure.
- Database operations (CRUD) shall execute within acceptable time limits.
- Maintain **99% uptime availability**.

3.4 Logical database requirements

- Maintain separate tables/collections for Users, Subscriptions, Orders, Payments, Routes, and Reports.
- Each user shall have a **unique user ID**.
- Subscriptions linked to specific customers; Orders linked to subscriptions and partners.
- Ensure data integrity, consistency, and referential relationships.
- Backup and recovery mechanisms supported.

3.5 Design Constraints

- Developed as a **web-based application**.
- Use relational or NoSQL database (e.g., MongoDB).
- Integrate with third-party payment gateway and map APIs.
- Role-based access control must be enforced.
- Operate over standard **HTTP/HTTPS protocols**.
- Compatible with modern web browsers and depends on stable internet connectivity.

3.6 Software System attributes

3.6.1 Reliability

- Operates without failure during normal usage.
- Data stored accurately without loss or corruption.
- Graceful error handling with proper messages.

3.6.2 Availability

- Available **24/7** for users.
- Maintain at least **99% uptime**.
- Scheduled maintenance communicated in advance.

3.6.3 Security

- User passwords shall be encrypted.
- Role-based access control enforced.
- Transactions use secure HTTPS protocol.
- Unauthorized access prevented; sensitive data protected.

3.6.4 Maintainability

- Modular design; code well-structured and documented.
- Database design allows easy updates.
- Future enhancements supported without major system changes.

3.6.5 Portability

- Accessible through modern web browsers.
- Support for desktop and mobile devices.
- Deployable on different server environments.

4. SUPPORTING INFORMATION

(Additional appendices, diagrams, or indices to be added as required)